

PROJECT REPORT

On

EXAM-AI

Submitted by

Patel Dhairya (IU2341230041)

Khatri Keshav (IU2341230068)

Patel Swet (IU2341230111)

In fulfillment for the award of the degree

Of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING



**INSTITUTE OF TECHNOLOGY AND ENGINEERING
INDUS UNIVERSITY CAMPUS, RANCHARDA, VIA-THALTEJ
AHMEDABAD-382115, GUJARAT, INDIA,**

WEB: www.indusuni.ac.in

OCTOBER 2026

PROJECT REPORT

ON

EXAM-AI

AT



In the partial fulfillment of the requirement
for the degree of
Bachelor of Technology
in
Computer Science And Engineering

PREPARED BY

PATEL DHAIRYA(IU2341230041)
KHATRI KESHAV(IU2341230068)
PATEL SWET(IU2341230111)

UNDER GUIDANCE OF

Internal Guide
Ms. Shreya Bhatt
Assistant Professor,
Department of Computer Science and Engineering,
I.I.T.E, Indus University ,Ahmedabad

SUBMITTED TO

INSTITUTE OF TECHNOLOGY AND ENGINEERING
INDUS UNIVERSITY CAMPUS, RANCHARDA, VIA-THALTEJ
AHMEDABAD-382115, GUJARAT, INDIA,

WEB: www.indusuni.ac.in

OCTOBER 2026

CANDIDATE'S DECLARATION

I declare that the final semester report entitled “**EXAM-AI**” is my own work conducted under the supervision of the guide **Ms. Shreya Bhatt**.

I further declare that to the best of my knowledge, the report for B.Tech final semester does not contain part of the work which has been submitted for the award of B.Tech Degree either in this university or any other university without proper citation.

Candidate's Signature

PATEL DHAIRYA (IU2341230041)

Guide : Ms. Shreya Bhatt
Assistant Professor
Department of Computer Science And Engineering,
Indus Institute of Technology and Engineering
INDUS UNIVERSITY– Ahmedabad,
State: Gujarat

CANDIDATE'S DECLARATION

I declare that the final semester report entitled “**EXAM-AI**” is my own work conducted under the supervision of the guide **Ms. Shreya Bhatt**.

I further declare that to the best of my knowledge, the report for B.Tech final semester does not contain part of the work which has been submitted for the award of B.Tech Degree either in this university or any other university without proper citation.

Candidate's Signature

KHATRI KESHAV (IU2341230068)

Guide : Ms. Shreya Bhatt
Assistant Professor
Department of Computer Science And Engineering,
Indus Institute of Technology and Engineering
INDUS UNIVERSITY– Ahmedabad,
State: Gujarat

CANDIDATE'S DECLARATION

I declare that the final semester report entitled “**EXAM-AI**” is my own work conducted under the supervision of the guide **Ms. Shreya Bhatt**.

I further declare that to the best of my knowledge, the report for B.Tech final semester does not contain part of the work which has been submitted for the award of B.Tech Degree either in this university or any other university without proper citation.

Candidate's Signature

PATEL SWET (IU2341230111)

Guide : Ms. Shreya Bhatt
Assistant Professor
Department of Computer Science And Engineering,
Indus Institute of Technology and Engineering
INDUS UNIVERSITY– Ahmedabad,
State: Gujarat

INDUS INSTITUTE OF TECHNOLOGY AND ENGINEERING
COMPUTER SCIENCE AND ENGINEERING
2026 -2027



CERTIFICATE

Date: 01/10/2026

This is to certify that the project work entitled “**EXAM-AI**” has been carried out by **PATEL DHAIRYA** under my guidance in partial fulfillment of degree of Bachelor of Technology in **COMPUTER SCIENCE AND ENGINEERING (Final Year)** of Indus University, Ahmedabad during the academic year 2026- 2027.

Ms. Shreya Bhatt
Assistant Professor,
Department of Computer
Science And Engineering,
IITE, Indus University,
Ahmedabad

Dr. Kaushal Jani
Head of Department,
Department of Computer
Science And Engineering,
IITE, Indus University,
Ahmedabad

Prof. Zalak Vyas
Head of Department,
Department of Computer
Science And Engineering,
IITE, Indus University,
Ahmedabad

INDUS INSTITUTE OF TECHNOLOGY AND ENGINEERING
COMPUTER SCIENCE AND ENGINEERING
2026 -2027



CERTIFICATE

Date: 01/10/2026

This is to certify that the project work entitled “**EXAM-AI**” has been carried out by **KHATRI KESHAV** under my guidance in partial fulfillment of degree of Bachelor of Technology in **COMPUTER SCIENCE AND ENGINEERING (Final Year)** of Indus University, Ahmedabad during the academic year 2026- 2027.

Ms. Shreya Bhatt
Assistant Professor,
Department of Computer
Science And Engineering,
IITE, Indus University,
Ahmedabad

Dr. Kaushal Jani
Head of Department,
Department of Computer
Science And Engineering,
IITE, Indus University,
Ahmedabad

Prof. Zalak Vyas
Head of Department,
Department of Computer
Science And Engineering,
IITE, Indus University,
Ahmedabad

INDUS INSTITUTE OF TECHNOLOGY AND ENGINEERING
COMPUTER SCIENCE AND ENGINEERING
2026 -2027



CERTIFICATE

Date: 01/10/2026

This is to certify that the project work entitled “**EXAM-AI**” has been carried out by **PATEL SWET** under my guidance in partial fulfillment of degree of Bachelor of Technology in **COMPUTER SCIENCE AND ENGINEERING (Final Year)** of Indus University, Ahmedabad during the academic year 2026- 2027.

Ms. Shreya Bhatt
Assistant Professor,
Department of Computer
Science And Engineering,
IITE, Indus University,
Ahmedabad

Dr. Kaushal Jani
Head of Department,
Department of Computer
Science And Engineering,
IITE, Indus University,
Ahmedabad

Prof. Zalak Vyas
Head of Department,
Department of Computer
Science And Engineering,
IITE, Indus University,
Ahmedabad

ACKNOWLEDGEMENT

Towards the successful completion of my B.Tech in Computer Science and Engineering final year project training in Indus University, I feel greatly obliged to certain Specials.

I am sincerely grateful to Shreya Bhatt of Indus University, Ahmedabad, who provided me this opportunity and necessary support during the development of the project. I am also grateful to all staff members of this company for providing all the technical assistance and helping in solving problems arising during development.

I have no other words to express my sincere thanks to all the faculty of Indus University, Ahmedabad, for their kind cooperation and able guidance. Especially to Shreya Bhatt , who helped us in identifying the project definition. I would also like to thank Shreya Bhatt, my internal project guide in college, without whom the project could not be executed.

- Patel Dhairya
IU2341230041
Computer Science & Engineering

- Khatri Keshav
IU2341230068
Computer Science & Engineering

- Patel Swet
IU2341230111
Computer Science & Engineering

TABLE OF CONTENTS

1	Introduction.....	1
1.1	Project Summary.....	1
1.2	Project Purpose	1
1.3	Project Scope	1
1.4	Objectives.....	1
1.4.1	Main Objectives.....	1
1.4.2	Secondary Objectives.....	2
1.5	Technology and Literature Overview	2
1.5.1	Retrieval-Augmented Generation.....	2
1.5.2	Vector Search and Embeddings.....	2
1.5.3	Web Application Technologies.....	2
1.5.4	Literature Overview	2
1.6	Synopsis	2
2	Literature Survey.....	3
2.1	Introduction of Survey.....	3
2.2	Why Survey?.....	3
2.3	Existing System Versus Proposed System	3
3	Project Management	4
3.1	Project Planning Objectives.....	4
3.2	Project Scheduling	4
3.2.1	Basic Principle	4
3.2.2	Compartmentalization.....	4
3.2.3	Work Breakdown Structure.....	4
3.2.4	Project Organization.....	4
3.2.5	Time Allocation.....	5
3.3	Risk Management.....	5
4	System Requirements.....	6
4.1	User Characteristics.....	6
4.2	Functional Requirements	6
4.3	Activity and Proposed System.....	6
4.4	Non-Functional Requirements	6
4.5	Hardware and Software Requirement.....	7
5	System Analysis.....	8
5.1	Study of Current System.....	8
5.2	Problems in Current System.....	8
5.3	Requirement of New System	8
5.4	Process Model	8
5.5	Feasibility Study.....	8
5.5.1	Technical Feasibility.....	8
5.5.2	Operational Feasibility	8

5.5.3 Economical Feasibility	9
5.5.4 Schedule Feasibility.....	9
5.6 Features of New System.....	9
6 Testing	10
6.1 Black-Box Testing.....	10
6.2 White-Box Testing	10
6.3 Test Cases.....	10
7 System Design	11
7.1 System Architecture	11
7.2 Use-Case Model	11
7.3 Sequence Diagram: RAG Chat	13
7.4 Activity Diagram: Material Ingestion	13
7.5 Data Flow Diagram	15
7.5.1 Level 0 DFD.....	15
7.5.2 Level 1 DFD.....	15
7.6 Data Model.....	16
8 Result and Discussion.....	17
8.1 Student Study Features	17
8.2 Teacher Material and Quiz Features	17
8.3 Analytics and Tracking Features	17
8.4 Security and Reliability Results	19
8.5 Overall System Performance.....	19
9 Limitation and Future Enhancement	20
9.1 Limitations	20
9.2 Future Enhancement	20
10 Conclusion	21
10.1 Conclusion.....	21
11 Appendices.....	22
11.1 Business Model	22
11.2 Product Deployment Detail	22
11.3 API and Web Service Details.....	22
11.4 Repository Structure.....	22
12 Bibliography.....	23

ABSTRACT

ExamAI is a full-stack, AI-powered exam-preparation platform designed to connect students with teacher-approved learning material. The platform combines retrieval-augmented generation (RAG) chat, teacher-authored shared quizzes, AI-assisted quiz generation, and student-owned flashcard decks in one web application. It provides separate workflows for students and teachers while keeping authorization, material ownership, and academic comparability explicit in the system design.

Students can select a subject, choose the approved materials that should scope a study session, ask questions, receive answers with teacher and filename attribution, attempt published quizzes, review weak topics, and generate flashcards. Teachers can upload PDF, PPTX, and DOCX materials, monitor ingestion status, author or generate quizzes, publish shared assessments, and review class-wide analytics.

The backend uses FastAPI, SQLAlchemy, Alembic, Supabase Postgres and Auth, Gemini, Qdrant Cloud, and local sentence-transformer embeddings. The frontend uses React, Vite, React Router, Zustand, Tailwind CSS, and reusable interface primitives. The project demonstrates secure role-based access, metadata-filtered retrieval, concurrent ingestion protection, structured-output retry handling, server-side quiz grading, and measured frontend performance. The current implementation contains 82 offline backend tests and 63 frontend tests.

The report presents the motivation, objectives, literature and technology overview, project management approach, requirements, system analysis, testing, system design, results, limitations, future enhancements, and conclusion of the ExamAI project.

LIST OF FIGURES

Figure	Title	Page
Figure 7.1	System architecture	11
Figure 7.2	Use-case model	12
Figure 7.3	RAG chat sequence	13
Figure 7.4	Material ingestion activity	15
Figure 7.5	Data flow overview	15
Figure 8.1	Student study workflow	17
Figure 8.2	Teacher analytics workflow	19

LIST OF TABLES

Table	Title	Page
Table 2.1	Existing and proposed system comparison	3
Table 3.1	Work breakdown structure	4
Table 3.2	Time allocation	5
Table 3.3	Risk management	5
Table 4.1	Functional requirements	6
Table 4.2	Non-functional requirements	6
Table 6.1	Representative test cases	10

ABBREVIATIONS

API	Application Programming Interface
AI	Artificial Intelligence
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DFD	Data Flow Diagram
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
SQL	Structured Query Language
UI	User Interface
UX	User Experience
UUID	Universally Unique Identifier
JWT	JSON Web Token
PPTX	PowerPoint Open XML Presentation
PDF	Portable Document Format

1 Introduction

1.1 Project Summary

ExamAI is a web-based exam-preparation platform that supports two roles: student and teacher. Students use the system to study from teacher-approved materials, ask contextual questions, attempt common subject quizzes, and revise through AI-generated flashcards. Teachers use the system to manage subject material, create assessments, and monitor class performance. The design treats learning support and assessment as related but distinct activities: RAG chat and flashcards are student study tools, whereas published quizzes are shared teacher-authored assessments.

The platform is implemented as a React frontend and a FastAPI backend. Supabase provides authentication, PostgreSQL data storage, and private file storage. Qdrant stores embedded material chunks for metadata-filtered retrieval. Gemini provides structured language-model generation for chat responses, quizzes, and flashcards. The application is intended as a stable academic prototype with a clear path for future deployment and extension.

1.2 Project Purpose

The purpose of ExamAI is to reduce the fragmentation of exam preparation. Students commonly consult multiple files, search manually for explanations, and receive limited feedback about weak topics. Teachers also need a practical way to distribute approved material, create comparable assessments, and inspect class-wide progress. ExamAI brings these activities into one role-aware system.

A central purpose of the project is to make AI assistance traceable. The system does not treat a generated answer as sufficient by itself. Each retrieved source carries material, teacher, and page or slide metadata. The returned answer therefore exposes numbered citations that allow a student to identify the source material and the teacher responsible for it.

1.3 Project Scope

The scope includes email/password login for seeded users, student enrollment and teacher subject membership, teacher-owned material uploads, PDF/PPTX/DOCX ingestion, subject-scoped RAG chat, teacher-authored and AI-assisted quizzes, server-side grading, student-owned flashcard generation, and separate teacher analytics views.

The current scope excludes student-generated graded quizzes, in-app teacher–student messaging, a permanent student material-selection table, an administrator role, payment workflows, mobile applications, and clinical or professional decision-making. These boundaries protect the fairness of shared assessment and keep the implementation focused on the academic use case.

1.4 Objectives

1.4.1 Main Objectives

- Develop a secure web platform for AI-assisted exam preparation.
- Provide attributable RAG answers over teacher-approved material.
- Support shared teacher-authored quizzes with comparable class results.
- Generate personal flashcard decks from material selected by a student.
- Provide teacher dashboards for quiz analytics and student progress.

- Maintain a modular, testable backend and a responsive frontend.

1.4.2 Secondary Objectives

- Enforce subject membership and role authorization in service-layer logic.
- Preserve source metadata throughout parsing, chunking, embedding, retrieval, and citation.
- Handle structured LLM failures through schema-driven, error-aware retries.
- Protect ingestion from concurrent upload, retry, and delete operations.
- Measure frontend performance rather than relying only on subjective responsiveness.

1.5 Technology and Literature Overview

1.5.1 Retrieval-Augmented Generation

Retrieval-augmented generation combines information retrieval with language generation. Instead of asking a model to answer from general parameters alone, the system retrieves relevant chunks from a controlled collection and supplies them as context. ExamAI uses this approach for subject chat and AI-assisted quiz generation. The retrieval layer is constrained by subject membership and selected material identifiers before the vector database is queried.

1.5.2 Vector Search and Embeddings

An embedding represents text as a numerical vector so that semantically related passages can be compared. ExamAI uses local `all-MiniLM-L6-v2` sentence embeddings and a single Qdrant collection. Payload metadata includes `subject_id`, `material_id`, `teacher_id`, `filename`, `teacher name`, and `source locator`. The single-collection strategy allows metadata filtering without wiping or recreating a collection for each request.

1.5.3 Web Application Technologies

FastAPI provides the HTTP API and keeps route handlers thin. SQLAlchemy models represent persistent entities, while Pydantic schemas define API and LLM contracts. React and Vite provide the client interface. Supabase supports authentication, PostgreSQL, and private storage. Alembic manages schema migrations. Render.com is the intended backend deployment environment.

1.5.4 Literature Overview

Existing learning platforms commonly provide content management, quizzes, or recommendation features, but these functions are often separated across tools. Research and practice in educational technology show the value of immediate feedback, retrieval practice, and targeted revision. ExamAI combines these principles with source attribution and teacher ownership. The project also treats shared quizzes as a class-comparability mechanism rather than generating an unrelated assessment for every student.

1.6 Synopsis

ExamAI provides a unified but role-separated study environment. Its most important design decisions are teacher-owned materials, session-level material selection, attributable retrieval, shared published quizzes, and student-owned flashcards. The subsequent chapters explain how these decisions were translated into requirements, architecture, implementation, testing, and evaluation.

2 Literature Survey

2.1 Introduction of Survey

The survey considers the operational problem addressed by ExamAI rather than attempting to reproduce a general review of artificial intelligence. The comparison focuses on content ownership, retrieval traceability, assessment consistency, feedback, analytics, and security. These criteria are important because an exam-preparation system must support both individual learning and teacher oversight.

2.2 Why Survey?

The survey was conducted to identify limitations in conventional exam-preparation workflows and to define the boundary of the proposed system. Manual file searching can be slow and can produce answers without clear evidence. Independent quiz links may not provide comparable class data. A system that combines an unrestricted model with unapproved material can also reduce trust. The survey therefore informed the use of teacher-approved sources, shared quizzes, and explicit citations.

2.3 Existing System Versus Proposed System

Criterion	Existing or conventional approach	ExamAI approach
Source control	Files are searched manually or across mixed sources	Teacher-owned, subject-scoped and status-checked material
Question answering	Generic search or unsupported model response	RAG answer scoped to selected material with numbered citations
Assessment	Separate or individually authored quizzes	One published quiz per topic for all students in the subject
Feedback	Scores may be available without topic analysis	Per-question feedback and weak-topic detection
Revision	Static notes or manually created cards	Student-generated flashcard decks
Teacher insight	Limited or scattered reporting	Quiz analytics and cross-quiz progress read models
Security	Role checks may be inconsistent	Service-layer membership authorization and secure session handling

The comparison shows that ExamAI is not only a chatbot. It is a role-aware academic workflow in which retrieval, assessment, and analytics are connected by explicit data and authorization rules.

3 Project Management

3.1 Project Planning Objectives

The project was planned as a sequence of stable feature phases. Each phase was intended to leave a working build ahead of the next review milestone. Planning objectives were to keep the architecture understandable, validate risky integrations early, preserve a conservative version, and document decisions that affected later features.

3.2 Project Scheduling

3.2.1 Basic Principle

Development followed an incremental approach. Foundations were established before dependent features: authentication and data access preceded material management; material ingestion preceded retrieval; retrieval preceded AI-assisted quiz generation and flashcards; shared quizzes preceded analytics. This order reduced rework because later features could use stable service contracts.

3.2.2 Compartmentalization

The work was divided into eight feature phases, numbered Phase 0 through Phase 7, followed by cleanup and hardening work. Backend, frontend, integration, testing, and documentation concerns were tracked together for each phase rather than treated as unrelated projects.

3.2.3 Work Breakdown Structure

Phase	Primary work	Main outcome
0	Foundations	Authentication, schema, project conventions
1	Subject and material access	Roles, membership, teacher-owned materials
2	Ingestion	PDF/PPTX/DOCX parsing, chunking and embeddings
3	RAG chat	Filtered retrieval and citations
4	Quizzes	Manual and AI-assisted authoring, grading
5	Flashcards	Student-generated study decks
6	Analytics and performance	Teacher reports and frontend optimization
7	Hardening	Security, quality, tests and cleanup

3.2.4 Project Organization

The project is organized as an academic software-development effort with connected technical, managerial, documentation, and evaluation activities. The work includes problem definition, stakeholder and user analysis, requirements review, architecture and data-model design, backend and frontend implementation, integration with external services, testing, security review, performance measurement, demonstration preparation, and report writing. The backend owns authorization, persistence, ingestion, retrieval, generation, grading, and analytics. The frontend owns navigation, session state, forms, loading

states, caching, and presentation. Shared contracts are defined through API schemas and response envelopes. Progress is reviewed against milestones, risks are recorded before they become blockers, and each completed phase is accompanied by a learning walkthrough and evidence of testing.

3.2.5 Time Allocation

Work area	Relative allocation	Description
Requirements and design	20%	Scope, data model, architecture, interaction flows
Backend implementation	30%	Routes, services, ingestion, AI and analytics
Frontend implementation	25%	Pages, state, API client, performance behavior
Testing and hardening	15%	Backend and frontend tests, security, cleanup
Documentation and review	10%	Walkthroughs, reports, demonstrations and revisions

3.3 Risk Management

Risk	Potential effect	Mitigation
LLM model or quota changes	Generation features become unavailable	Verify the live model early; use schemas and error-aware retries
Incorrect retrieval scope	Answers cite unrelated material	Validate material IDs in Postgres and filter Qdrant by subject and material
Concurrent ingestion operations	Deleted or stale rows may reappear	Per-material locks, row locks and conditional status updates
Unclear role boundaries	Unauthorized content access	Service-layer membership checks for every subject-scoped operation
Slow external services	Poor request responsiveness	Thread offloading, bounded caching and request deduplication
Scope expansion	Unstable delivery	Explicitly defer personalized quizzes and messaging

4 System Requirements

4.1 User Characteristics

Students are expected to understand basic web navigation and to have access to a browser. They can select enrolled subjects, choose approved learning material, ask questions, attempt published quizzes, and review personal study content. Teachers are expected to manage subject material and assessments. They can upload supported files, monitor processing, author or edit quizzes, publish them, and inspect analytics. The system does not assume that users understand vector search or LLM operation.

4.2 Functional Requirements

ID	Requirement
FR-01	The system shall authenticate a seeded user with email and password and derive the role from the backend profile.
FR-02	The system shall allow teachers to upload PDF, PPTX and DOCX materials for a subject.
FR-03	The system shall process uploaded material and expose processing, ready and failed status.
FR-04	The system shall allow enrolled students to select ready materials for a study session.
FR-05	The system shall return RAG answers with numbered citations containing teacher and filename attribution.
FR-06	The system shall allow teachers to create manual or AI-assisted quiz drafts.
FR-07	The system shall require teacher publication before a quiz is available to students.
FR-08	The system shall grade attempts on the server and return feedback and weak topics.
FR-09	The system shall generate personal flashcard decks from selected material.
FR-10	The system shall provide quiz analytics and cross-quiz student progress views.

4.3 Activity and Proposed System

A typical student activity begins with login and subject selection. The student chooses a session material scope, asks a question, and receives an answer grounded in retrieved chunks. The student may then take a published quiz, review feedback, and generate flashcards from the same or another selected material set. A teacher activity begins with subject access, continues through material upload and quiz authoring, and ends with publication and analytics review.

4.4 Non-Functional Requirements

Category	Requirement
Security	Role and membership authorization shall be enforced in services; access tokens shall not be stored in localStorage; refresh tokens shall use an HttpOnly cookie.
Reliability	The system shall handle malformed structured model output, failed ingestion, retry, delete and concurrent operations without corrupting state.

Performance	Blocking ingestion work shall not occupy the event loop; the frontend shall use route splitting, bounded GET caching, request deduplication and stale-response protection.
Maintainability	Routes shall remain thin, schemas shall remain separate from ORM models, and business logic shall reside in services.
Usability	The interface shall expose loading, empty, failure and success states and shall make material scope visible to the student.
Traceability	Every citation shall retain enough metadata to identify the teacher, file and source location.

4.5 Hardware and Software Requirement

The development environment requires a modern computer with at least 8 GB RAM, a current browser, Python 3.14, Node.js 20.19 or later, and network access to configured Supabase, Qdrant, and Gemini services. The backend uses FastAPI, SQLAlchemy, Alembic, Pydantic, sentence-transformers, Qdrant client, and Supabase libraries. The frontend uses React, Vite, React Router, Zustand, Tailwind CSS, and testing tools based on Vitest and Testing Library.

5 System Analysis

5.1 Study of Current System

The conventional workflow is distributed across lecture files, search engines, messaging channels, separate quiz tools, and personal notes. A student may read several documents before finding a relevant explanation. A teacher may distribute material in one location, collect quiz responses in another, and manually interpret performance. These steps provide limited continuity between study activity and assessment.

5.2 Problems in Current System

The current workflow creates five problems. First, source identification is weak when answers are not connected to a page, slide, or teacher. Second, material access is difficult to control when files are copied without a subject-level permission model. Third, separate quiz forms reduce class-wide comparability. Fourth, feedback often stops at a total score. Fifth, creating revision material requires additional manual work.

5.3 Requirement of New System

The proposed system requires teacher ownership, subject-level authorization, source-aware retrieval, shared assessment, personal revision content, and analytics. These requirements are represented in the data model through subjects, teacher membership, student enrollment, materials, quizzes, attempts, and flashcard decks. The material selection used for chat or flashcard generation is intentionally request-time data rather than a permanent student preference.

5.4 Process Model

ExamAI follows an incremental software-development process. Requirements are converted into data and service contracts. A vertical slice is then implemented across the backend and frontend, tested offline where possible, and documented in a walkthrough. Risks that depend on external services are isolated behind utility and service boundaries. Cleanup passes are performed after features stabilize.

5.5 Feasibility Study

5.5.1 Technical Feasibility

The system is technically feasible because the selected technologies support the required functions and have clear interfaces. FastAPI and React provide a conventional web boundary. PostgreSQL handles relational authorization and assessment data. Qdrant supports filtered vector retrieval. Gemini supports structured generation when its output is validated with Pydantic schemas. The main technical uncertainty is external model availability and quota, which is handled through configuration and error-aware generation paths.

5.5.2 Operational Feasibility

The proposed workflows match the responsibilities of students and teachers. Students see approved material and personal progress. Teachers retain control over source files and shared quizzes. The system is therefore operationally suitable for a classroom prototype, although deployment would require institution-specific onboarding, support, backup, and privacy procedures.

5.5.3 Economical Feasibility

The prototype uses open-source frameworks and managed free or development-tier services where appropriate. Local embeddings reduce repeated remote embedding costs. The principal variable costs are hosted database, vector storage, file storage, and model usage. The architecture allows these services to be replaced or scaled independently.

5.5.4 Schedule Feasibility

The phased plan is schedule-feasible because the project keeps a stable build ahead of review milestones and defers non-essential features. Student-generated graded quizzes, messaging, mobile applications, and advanced hybrid retrieval remain outside the current delivery boundary.

5.6 Features of New System

ExamAI provides secure login, subject dashboards, teacher-owned material management, document ingestion, attributable RAG chat, shared quizzes, AI-assisted quiz drafting, server-side grading, flashcard generation, quiz analytics, student progress tracking, and frontend performance safeguards. The features work through shared contracts rather than independent demonstrations.

6 Testing

6.1 Black-Box Testing

Black-box testing validates behavior through API and interface inputs without relying on internal implementation details. Representative cases include login success and failure, unauthorized subject access, material upload validation, ingestion retry, chat with an authorized scope, chat with an unrelated material identifier, quiz publication, duplicate attempt submission, flashcard generation, and analytics access.

6.2 White-Box Testing

White-box testing validates internal branches and service interactions. The backend suite covers authorization checks, schema validation, ingestion status transitions, retrieval filtering, citation mapping, structured-output retries, grading, analytics calculations, and response envelopes. The frontend suite covers route behavior, loading and error states, forms, API interactions, and user-visible state changes.

6.3 Test Cases

ID	Test case	Expected result	Status
TC-01	Valid student login	Access token/session established and role returned	Pass
TC-02	Student requests another subject's material	Service returns forbidden response	Pass
TC-03	Teacher uploads unsupported file	Validation error; file is not ingested	Pass
TC-04	Student selects unrelated material ID	Authorization fails before Qdrant query	Pass
TC-05	RAG answer generated	Answer includes numbered citations with teacher and filename	Pass
TC-06	Malformed LLM JSON	Error-aware retry validates the corrected output	Pass
TC-07	Student submits quiz twice	Attempt is idempotent and score is not duplicated	Pass
TC-08	Teacher reads analytics	Per-quiz and cross-quiz views return separate results	Pass
TC-09	Frontend stale response	Older response does not overwrite current state	Pass

The verified project context records 82 backend tests and 63 frontend tests as green. Backend tests run offline and complete in approximately 0.8 seconds; integration with hosted services remains environment-dependent.

7 System Design

7.1 System Architecture

The architecture has a presentation layer, an API and service layer, a relational data layer, a vector retrieval layer, and external AI and storage services. The frontend calls the FastAPI API directly. Route handlers authenticate the request and delegate to services. Services enforce membership, read or write relational state, and call ingestion, retrieval, generation, grading, or analytics components.

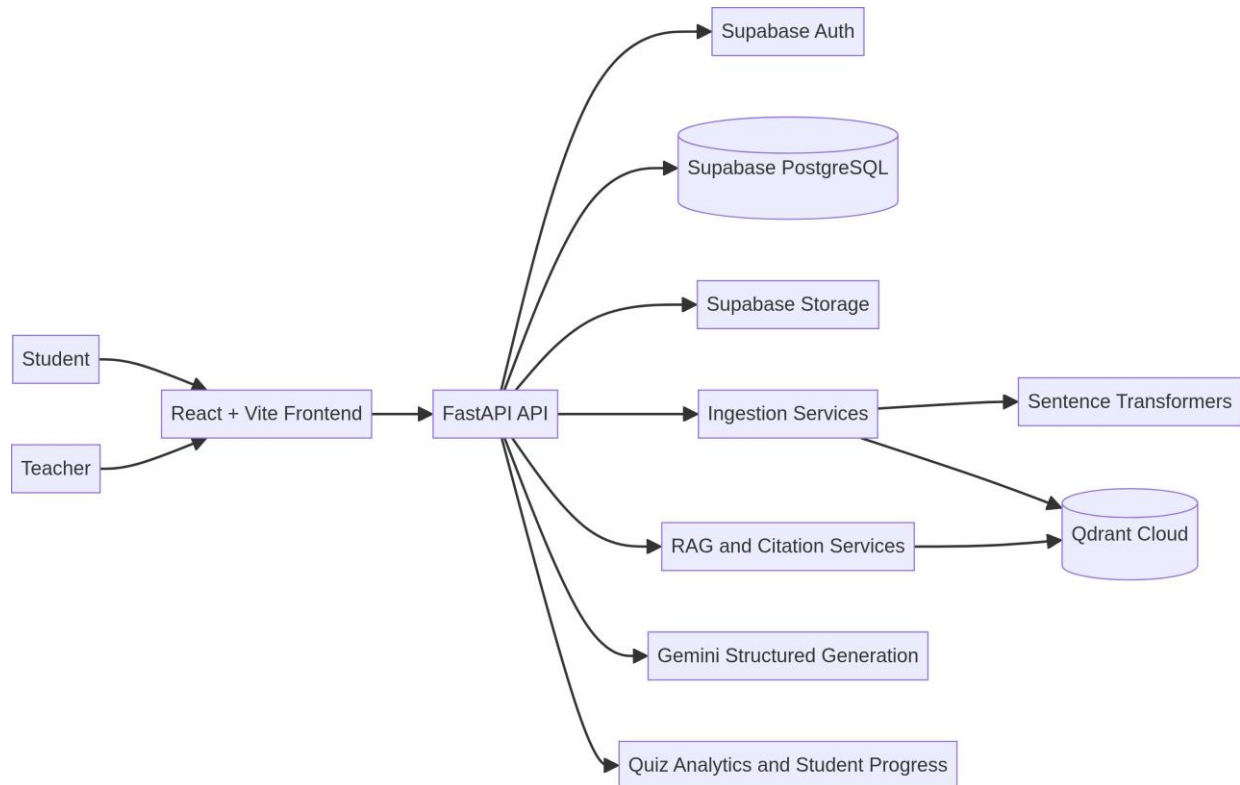


图 1 System architecture.

Layer	Responsibilities
Frontend	React pages, routing, state, API client, caching, loading and error states
API	Authentication, request validation, response envelope and request identifiers
Services	Materials, ingestion, RAG, quizzes, flashcards and analytics
Relational data	Users, subjects, memberships, materials, quizzes, attempts and decks
Vector and AI services	Qdrant retrieval, Gemini generation, local embeddings
Storage	Private teacher material files and application configuration

7.2 Use-Case Model

The student use cases are login, view subjects, select material, ask a question, take a quiz, review results, generate flashcards, and study a deck. The teacher use cases are login, view a subject, upload material, monitor ingestion, retry or delete material, author or generate a quiz, edit and publish a quiz, and view analytics. Both roles use the secure authentication boundary.

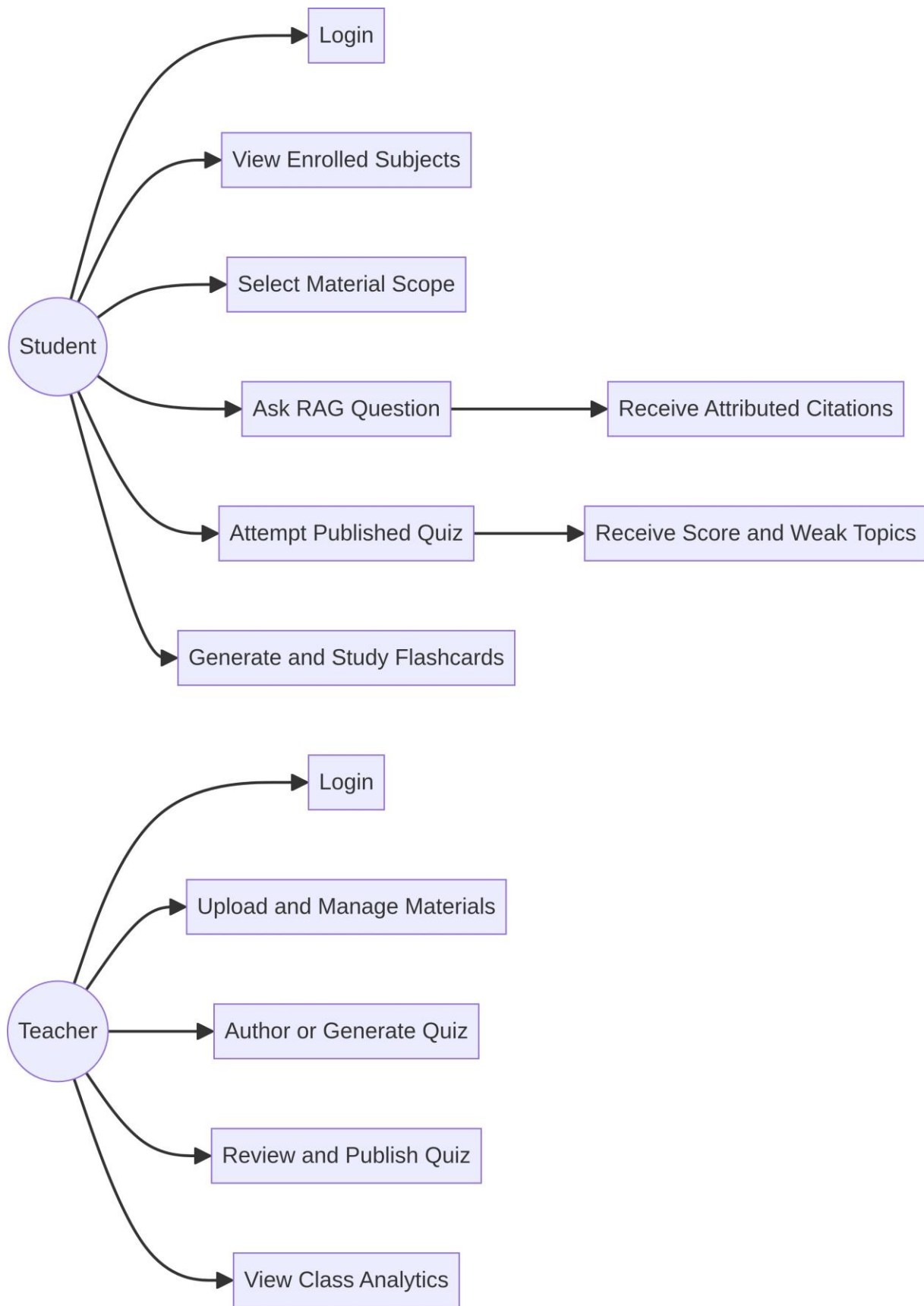


图 2 Use-case model.

7.3 Sequence Diagram: RAG Chat

The RAG sequence is: the frontend sends subject, selected material identifiers, and a question; the service checks enrollment and validates every identifier in PostgreSQL; the question is embedded; Qdrant is queried with subject and material filters; Gemini receives numbered context; the response is parsed; citation markers are mapped to payload metadata; and the API returns answer text plus citations.

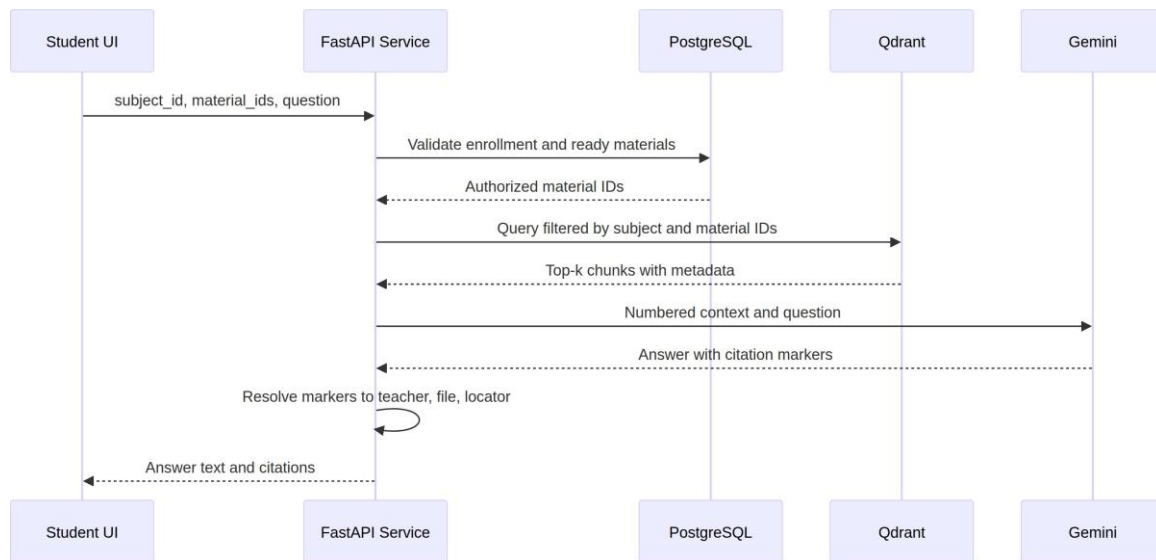


图 3 RAG chat sequence.

7.4 Activity Diagram: Material Ingestion

Material ingestion begins with a teacher upload. The service validates type and size, stores the private file, creates a processing record, parses the document, normalizes text and metadata, chunks content, creates embeddings, upserts points in batches, and marks the material ready. A failure marks the material failed and exposes retry. Version-safe status updates prevent stale work from reviving deleted records.

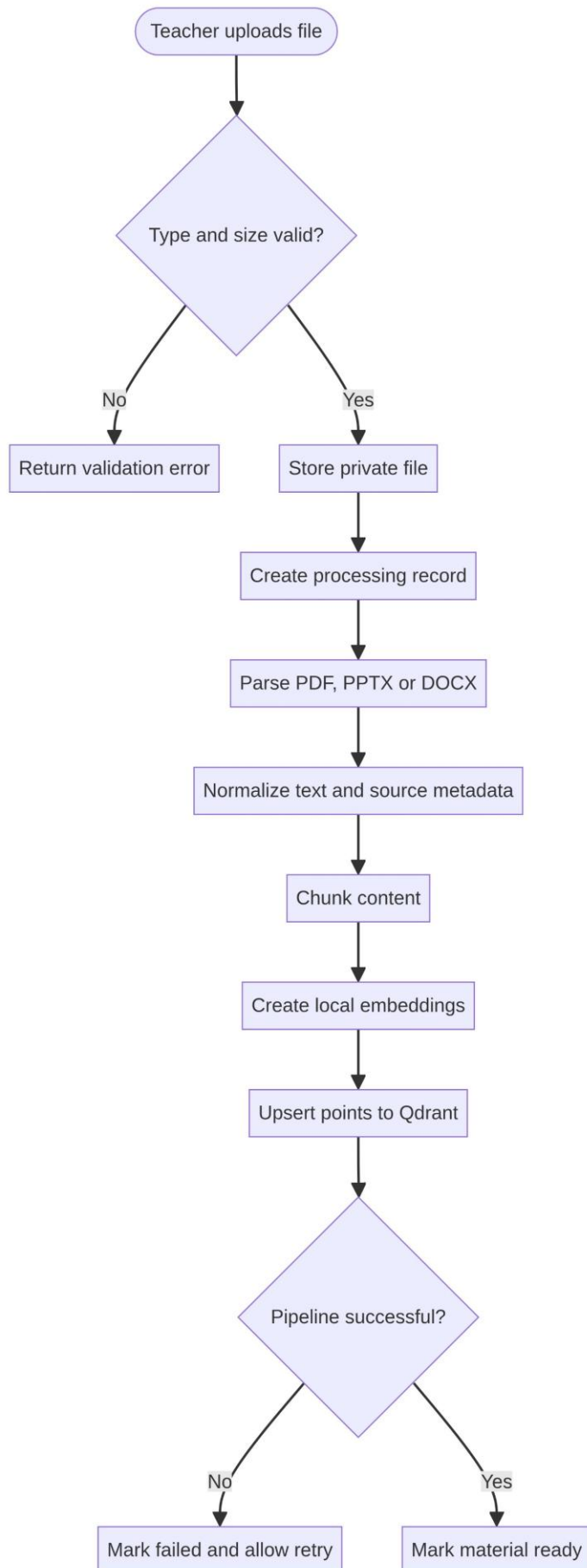


图 4 Material ingestion activity.

7.5 Data Flow Diagram

7.5.1 Level 0 DFD

At the highest level, students and teachers interact with ExamAI. ExamAI communicates with Supabase for identity, relational data, and file storage; Qdrant for vector retrieval; and Gemini for structured generation. The system returns study answers, assessments, feedback, decks, and analytics.

7.5.2 Level 1 DFD

The level-one flow separates authentication, subject access, material ingestion, RAG chat, quiz management, flashcard generation, and analytics. Authorization is a cross-cutting control. Material metadata flows from PostgreSQL to Qdrant payloads during ingestion and returns from Qdrant as citation metadata during chat.

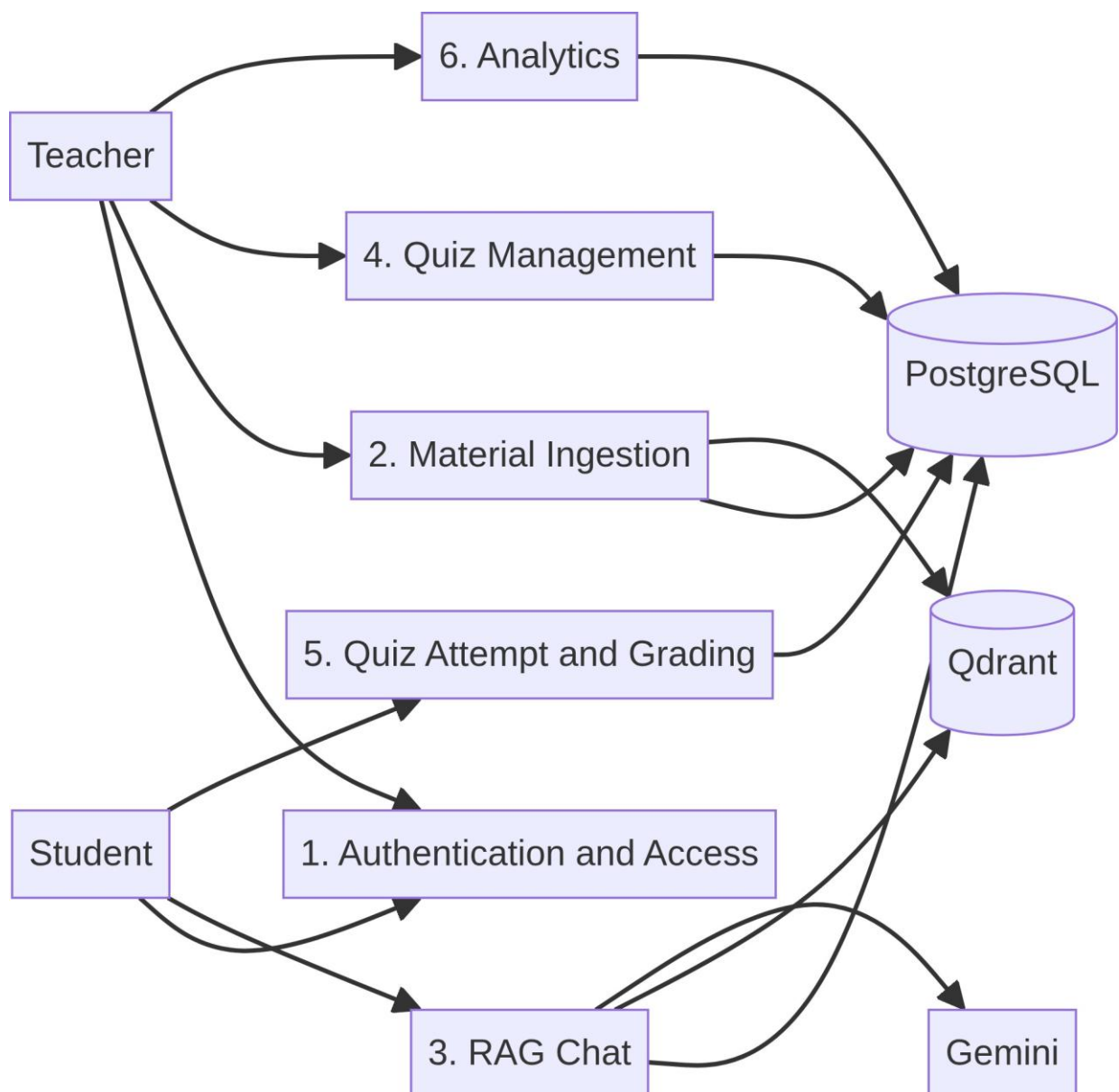


图 5 Data flow overview.

7.6 Data Model

The data model contains `users`, `subjects`, `subject_teachers`, `student_subjects`, `materials`, `quizzes`, `quiz_questions`, `quiz_attempts`, `flashcard_decks`, and `flashcards`. A material belongs to one teacher and one subject. A subject may have multiple teachers. A quiz is teacher-authored and subject-scoped. An attempt belongs to one student and one quiz. A flashcard deck belongs to one student and records the source material identifiers used for generation.

8 Result and Discussion

8.1 Student Study Features

The student dashboard presents enrolled subjects and quick statistics. The subject view is multi-teacher aware and groups materials by teacher. In RAG chat, a visible material-scope panel lets the student decide which ready materials should ground the session. Returned citations identify the teacher and filename, which improves traceability compared with an unattributed answer.

Students can access published quizzes and complete them under the configured time limit. The server calculates the result and returns per-question feedback and weak topics. Flashcards are personal rather than shared. A student can generate a deck from selected material and mark cards as still learning or mastered.

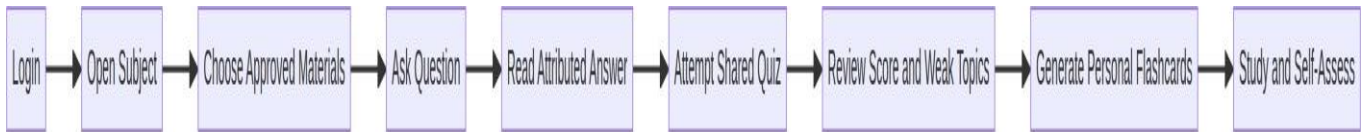


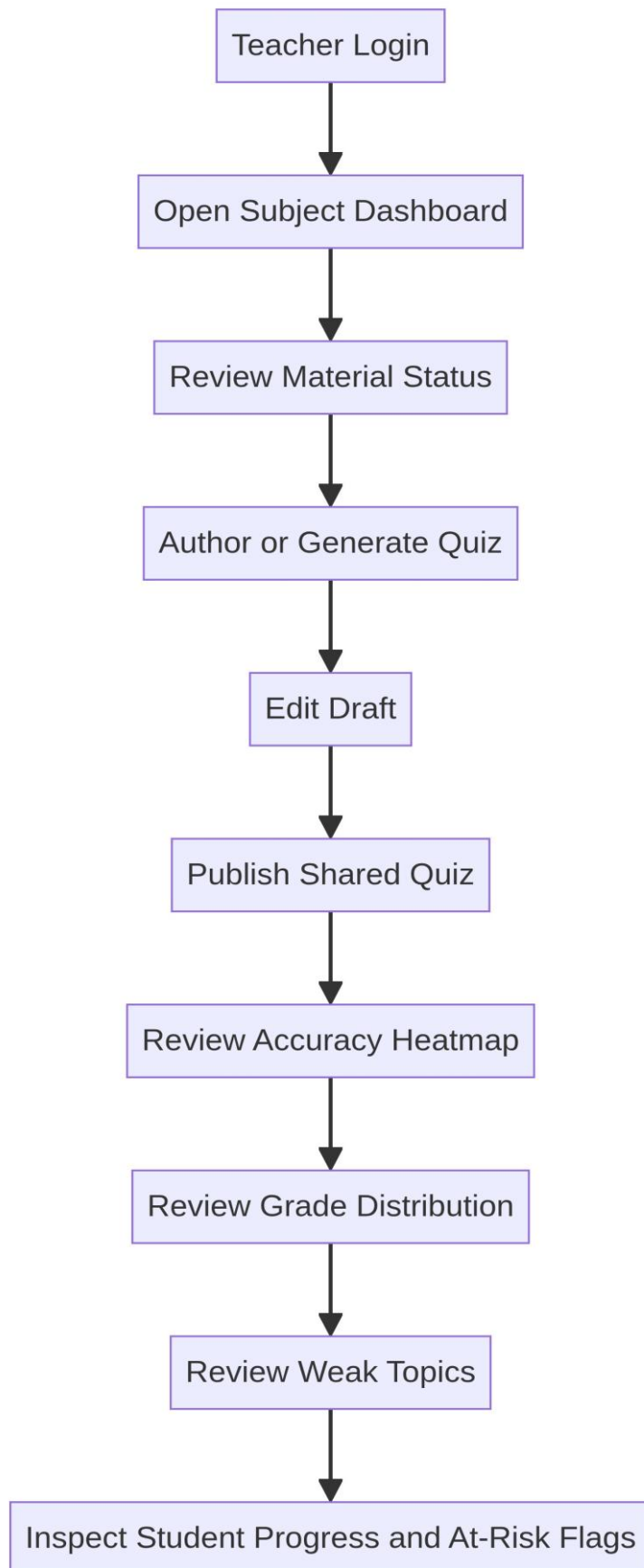
图 6 Student study workflow.

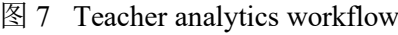
8.2 Teacher Material and Quiz Features

Teachers can upload PDF, PPTX, and DOCX files within the configured size limit. The interface shows processing, ready, and failed states and supports retry and deletion. Teachers can create quizzes manually or request AI-assisted generation from ready material. Both paths produce the same question shape. AI-generated quizzes remain drafts until a teacher reviews and publishes them.

8.3 Analytics and Tracking Features

The analytics design deliberately separates per-quiz analysis from cross-quiz student progress. Per-quiz analytics include a question-accuracy heatmap, grade distribution, and weak topics. Student progress includes average score, completion ratio, last activity, at-risk flagging, and drill-down information. Keeping these read models separate avoids forcing two different questions into one overloaded query.



图 7 Teacher analytics workflow.

8.4 Security and Reliability Results

The implementation keeps access tokens in memory and uses an HttpOnly refresh cookie scoped to the authentication path. The API client performs one silent refresh and replay after a 401 before redirecting to login. CORS uses an allow-list. Storage paths are not serialized to clients. Authorization checks are repeated in services rather than relying only on route declarations.

The ingestion pipeline uses per-material striped locks, row-level guards, and version-safe updates. Blocking parser and embedding calls are moved off the event loop. Structured LLM generation uses Pydantic schemas as the output contract and includes the error, bad response, and schema during retry. These patterns improve reliability without pretending that external AI services are deterministic.

8.5 Overall System Performance

The project context records 82 backend tests and 63 frontend tests as passing. Frontend performance work includes route-level code splitting, intentional navigation prefetching, bounded safe-GET caching, concurrent request deduplication, debounced search, loading skeletons, and stale-response handling. These measures target perceived and measured responsiveness while preserving correctness.

9 Limitation and Future Enhancement

9.1 Limitations

The current system is a capstone prototype and depends on configured external services. The live Gemini model name, quota, and billing configuration must be verified for each deployment. Hybrid dense-and-sparse retrieval is not part of the current Phase 1 decision. The system has no administrator role, mobile application, in-app messaging, or payment integration. The seed operation provisions users rather than offering public signup. Analytics are based on the application's quiz attempts and do not claim to measure long-term learning outcomes.

The generated answer is also limited by the quality and coverage of uploaded material. Source attribution identifies the retrieved passage but does not guarantee that the model's interpretation is correct. Teachers must review AI-assisted quizzes before publication. These constraints are important for responsible use.

9.2 Future Enhancement

Future work may add hybrid dense-and-sparse retrieval, richer source previews, material version history, institutional identity integration, administrator controls, mobile clients, accessibility audits, and deployment observability. Student-generated personal quizzes may be considered as an explicit v2 feature, separate from shared teacher-authored assessment. Additional evaluation could compare retrieval quality, citation correctness, quiz quality, and study outcomes on a controlled dataset.

10 Conclusion

10.1 Conclusion

ExamAI presents a unified web platform for teacher-controlled, AI-assisted exam preparation. It addresses the weaknesses of fragmented study workflows by combining attributable RAG chat, shared quizzes, feedback, personal flashcards, and teacher analytics. The system's value lies not only in the use of an LLM but in the surrounding controls: teacher-owned material, session-level scope, service-layer authorization, source metadata, shared assessment, and reliability safeguards.

The project demonstrates a complete full-stack implementation with FastAPI, React, PostgreSQL, Qdrant, Gemini, Supabase, local embeddings, structured schemas, and a tested API. The current implementation is suitable as an academic prototype and provides a maintainable foundation for later evaluation, deployment, and enhancement.

11 Appendices

11.1 Business Model

ExamAI is primarily an academic project. A future institutional model could provide the platform to departments or coaching organizations with teacher-managed subjects, private storage, and usage-based AI limits. Any such model would require clear privacy, data-retention, accessibility, and service-cost policies. The current report does not claim commercial deployment.

11.2 Product Deployment Detail

For local development, the backend is run from the `backend` directory with Python 3.14 and its canonical virtual environment. Database migrations are applied with Alembic, demo data is seeded through the project seed command, and the FastAPI service exposes its API and documentation. The frontend is started with Vite and uses the configured API base URL. A deployment configuration may host the FastAPI backend on Render.com, the frontend as a static build, and managed Supabase and Qdrant services.

11.3 API and Web Service Details

The API is organized around authentication, subjects, materials, chat, quizzes, flashcards, analytics, health, and current-user endpoints. Responses use a standardized envelope with request identification and timing headers. Important contracts include the chat request containing subject and material identifiers, the chat response containing answer text and citations, the quiz question schema, the attempt submission contract, and the flashcard deck response. The API should be documented through the generated OpenAPI interface and protected by the same authorization rules used by service methods.

11.4 Repository Structure

Path	Purpose
<code>backend/app/routes/</code>	Thin HTTP route handlers
<code>backend/app/services/</code>	Business logic for ingestion, RAG, quizzes, flashcards and analytics
<code>backend/app/models/</code>	SQLAlchemy persistence models
<code>backend/app/schemas/</code>	Pydantic API and structured-output schemas
<code>backend/migrations/</code>	Alembic database migrations
<code>frontend/src/</code>	React pages, components, store, API and tests
<code>backend/plan/</code>	Phase and cleanup planning documents
<code>backend/Walkthrough/</code>	Per-session learning and reference walkthroughs

12 Bibliography

- [1] FastAPI Documentation, “FastAPI framework, high performance, easy to learn, fast to code, ready for production,” <https://fastapi.tiangolo.com/>
- [2] React Documentation, “The library for web and native user interfaces,” <https://react.dev/>
- [3] Qdrant Documentation, “Qdrant vector database documentation,” <https://qdrant.tech/documentation/>
- [4] Supabase Documentation, “Supabase documentation,” <https://supabase.com/docs>
- [5] Google AI for Developers, “Gemini API documentation,” <https://ai.google.dev/gemini-api/docs>
- [6] SQLAlchemy Documentation, “SQLAlchemy Python SQL toolkit and Object Relational Mapper,” <https://docs.sqlalchemy.org/>
- [7] Alembic Documentation, “Alembic database migration tool for SQLAlchemy,” <https://alembic.sqlalchemy.org/>
- [8] Hugging Face, “Sentence Transformers documentation,” <https://www.sbert.net/>
- [9] Vite Documentation, “Vite next generation frontend tooling,” <https://vite.dev/>
- [10] Tailwind CSS Documentation, “Tailwind CSS utility-first CSS framework,” <https://tailwindcss.com/docs>